

Project Executable Files

Date	2 July 2026
Team ID	xxxxxx
Project Name	Rising Waters: A Machine Learning Approach to Flood Prediction
Maximum Marks	3 Marks

Project Executable Files

This section requires submission of all executable and deployable files for your project. Ensure all files are properly organized, named, and uploaded to the specified submission platform. The evaluator must be able to run or deploy your solution using the provided files and instructions.

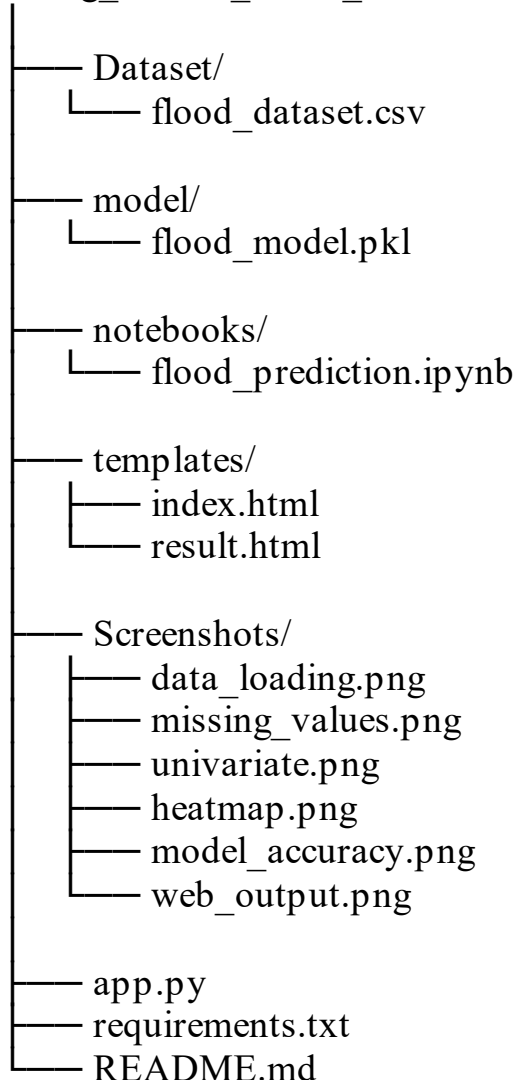
Step 1: Submission Checklist

S.No	Item to Submit	Submitted (Yes / No / NA)
1	Complete source code (all files and folders)	yes
2	README / Setup Guide (instructions to run the project)	yes
3	requirements.txt / package.json / dependency file	yes
4	Database schema / seed files (if applicable)	NA
5	Environment configuration file (.env.example or similar)	NA
6	Deployed application URL (if hosted)	NA
7	APK / Executable binary (if applicable for mobile/desktop apps)	NA
8	Dockerfile / Containerization config (if applicable)	NA
9	Test files and test results	Yes
10	Demo video or walkthrough (if required)	Yes

Step 2: File / Folder Structure

Provide an overview of your project's file and folder structure below:

Rising_Waters_Flood_Prediction/



Step 3: Deployment / Access Details

Field	Details
Hosted / Deployed URL	Not Deployed (Localhost Application)
Login Credential s (Demo)	Email : reddyprasanna271@gmail.com Password : yI1tdPuZ
Platform / Hosting	Local Flask Development Server

Provider	
Repository Link	https://github.com/7gdg45cdcr-maker/Rising-Waters-Flood-Prediction-ML
Demo Video Link	no

Step 4: Run Instructions

Provide clear, step-by-step instructions for the evaluator to run your project locally:

Step 1

Open Terminal and navigate to the project folder.

```
cd Rising_Waters_Flood_Prediction
```

Step 2

Install the required Python packages.

```
python3 -m pip install flask pandas numpy scikit-learn
```

Step 3

Run the Flask application.

```
python3 app.py
```

Step 4

The terminal displays:

```
* Serving Flask app 'app'
```

```
* Debug mode: on
```

```
* Running on http://127.0.0.1:5000
```

Step 5

Open the browser.

<http://127.0.0.1:5000>

Step 6

Enter the required values.

Example:

- Rainfall = 180
- Water Level = 8
- Humidity = 85

Click **Predict**.

Step 7

The application predicts the flood status.

Example Output:

Flood Likely

or
No Flood

Step 5: Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	Prediction accuracy depends on the training dataset quality.	Larger and real-time datasets can improve prediction accuracy.
2	The application currently runs only on a local Flask server.	It can be deployed on Render, Railway, or Heroku in future.
3	Only a limited number of environmental parameters are used.	Additional features such as soil moisture, wind speed, and temperature can be incorporated in future versions.